

Lesson 18: Project — Document Storage System

Learning Objectives

- Build a complete S3 + DynamoDB document storage system using Terraform
 - Create reusable modules for storage and metadata
 - Implement document indexing with DynamoDB
 - Apply all concepts learned so far
-

1. Architecture



Data Flow

1. Client uploads a document to S3
 2. Document metadata is stored in DynamoDB
 3. Each document has a unique ID that links the S3 object to its DynamoDB entry
-

2. Module: S3 Document Bucket

```
# modules/document-bucket/variables.tf
variable "environment" {
  description = "Deployment environment"
  type        = string
}

variable "bucket_name" {
  description = "Base name for the document bucket"
  type        = string
}

variable "enable_versioning" {
  description = "Enable versioning for document history"
  type        = bool
  default     = true
}

variable "force_destroy" {
```

```
description = "Allow deletion of non-empty bucket"
type        = bool
default     = false
}

variable "tags" {
  description = "Tags to apply"
  type        = map(string)
  default     = {}
}

# modules/document-bucket/main.tf
resource "aws_s3_bucket" "documents" {
  bucket          = var.bucket_name
  force_destroy   = var.force_destroy
  tags            = var.tags
}

resource "aws_s3_bucket_versioning" "documents" {
  bucket = aws_s3_bucket.documents.id
  versioning_configuration {
    status = var.enable_versioning ? "Enabled" : "Suspended"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "documents" {
  bucket = aws_s3_bucket.documents.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "documents" {
  bucket = aws_s3_bucket.documents.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}

# modules/document-bucket/outputs.tf
output "bucket_id" {
  value = aws_s3_bucket.documents.id
}

output "bucket_arn" {
  value = aws_s3_bucket.documents.arn
}

output "bucket_domain" {
```

```
value = aws_s3_bucket.documents.bucket_domain_name
}
```

3. Module: Document Index (DynamoDB)

```
# modules/document-index/variables.tf
variable "environment" {
  description = "Deployment environment"
  type        = string
}

variable "table_name" {
  description = "Name of the document index table"
  type        = string
}

variable "tags" {
  description = "Tags to apply"
  type        = map(string)
  default     = {}
}

# modules/document-index/main.tf
resource "aws_dynamodb_table" "index" {
  name           = var.table_name
  billing_mode   = "PAY_PER_REQUEST"
  hash_key       = "document_id"

  attribute {
    name = "document_id"
    type = "S"
  }

  global_secondary_index {
    name           = "filename-index"
    hash_key       = "filename"
    projection_type = "ALL"
  }

  attribute {
    name = "filename"
    type = "S"
  }

  tags = var.tags
}

# modules/document-index/outputs.tf
output "table_id" {
  value = aws_dynamodb_table.index.id
}
```

```
}

output "table_arn" {
  value = aws_dynamodb_table.index.arn
}
```

4. Root Configuration

```
# variables.tf
variable "environment" {
  description = "Deployment environment"
  type        = string
  default     = "dev"
}

variable "project_name" {
  description = "Project name"
  type        = string
  default     = "docstore"
}

# locals.tf
locals {
  bucket_name = "${var.project_name}-${var.environment}-documents"
  table_name  = "document-index-${var.environment}"
  tags = {
    Environment = var.environment
    Project      = var.project_name
    Service      = "document-storage"
  }
}

# main.tf
module "documents" {
  source = "../modules/document-bucket"

  environment      = var.environment
  bucket_name      = local.bucket_name
  enable_versioning = true
  force_destroy    = var.environment != "prod"
  tags             = local.tags
}

module "index" {
  source = "../modules/document-index"

  environment = var.environment
  table_name  = local.table_name
  tags        = local.tags
}
```

```
# outputs.tf
output "bucket_name" {
  value = module.documents.bucket_id
}

output "bucket_arn" {
  value = module.documents.bucket_arn
}

output "table_name" {
  value = module.index.table_id
}

output "table_arn" {
  value = module.index.table_arn
}

output "storage_endpoint" {
  value = "s3://${module.documents.bucket_id}"
}
```

5. Verification Commands

CLI — Verify Bucket

```
aws --endpoint-url=http://localhost:4566 s3 ls
aws --endpoint-url=http://localhost:4566 s3api get-bucket-versioning --
bucket docstore-dev-documents
aws --endpoint-url=http://localhost:4566 s3api get-bucket-encryption --
bucket docstore-dev-documents
```

CLI — Verify Table

```
aws --endpoint-url=http://localhost:4566 dynamodb list-tables
aws --endpoint-url=http://localhost:4566 dynamodb describe-table --table-
name document-index-dev
```

Simulate Document Upload

```
# Create a test document
echo "Hello, this is a test document" > /tmp/test-doc.txt

# Upload to S3
aws --endpoint-url=http://localhost:4566 s3 cp /tmp/test-doc.txt
```

```
s3://docstore-dev-documents/

# Record metadata in DynamoDB
aws --endpoint-url=http://localhost:4566 dynamodb put-item \
  --table-name document-index-dev \
  --item '{
    "document_id": {"S": "doc-001"},
    "filename": {"S": "test-doc.txt"},
    "size_bytes": {"N": "35"},
    "content_type": {"S": "text/plain"},
    "upload_date": {"S": "2026-06-09"}
  }'

# Query by document ID
aws --endpoint-url=http://localhost:4566 dynamodb get-item \
  --table-name document-index-dev \
  --key '{"document_id": {"S": "doc-001"}}'

# Query by filename (using GSI)
aws --endpoint-url=http://localhost:4566 dynamodb query \
  --table-name document-index-dev \
  --index-name filename-index \
  --key-condition-expression "filename = :f" \
  --expression-attribute-values '":{"f": {"S": "test-doc.txt"}}'
```

6. Key Takeaways

- S3 stores the raw documents; DynamoDB stores the metadata index
- Versioning on the bucket provides document history
- The DynamoDB GSI on filename enables search by filename
- Encryption ensures documents are encrypted at rest
- Public access blocking prevents unauthorized access
- The project uses two modules that each focus on one service
- Verification via CLI confirms the infrastructure works as expected

example and exercise repo url

<https://github.com/eyu-se/terraform-training-material-with-projects>

Trainer - Eyuel M.